The YOLO-Based Helmet Detection Pipeline

System Architecture Documentation

1. Introduction

Detecting whether a worker is wearing a safety helmet in a live video feed calls for an object detector that is both fast and accurate. This report documents such a system: a linear seven-stage pipeline built around a YOLO (You Only Look Once) model trained on a custom two-class dataset (helmet / no-helmet), where every incoming frame is prepared, passed through the network in a single forward pass, and the resulting detections are filtered, drawn, and returned.

The training data is a custom-built dataset combining publicly sourced Roboflow helmet images with additional custom-collected and manually annotated images, giving the model coverage of both general and site-specific conditions.

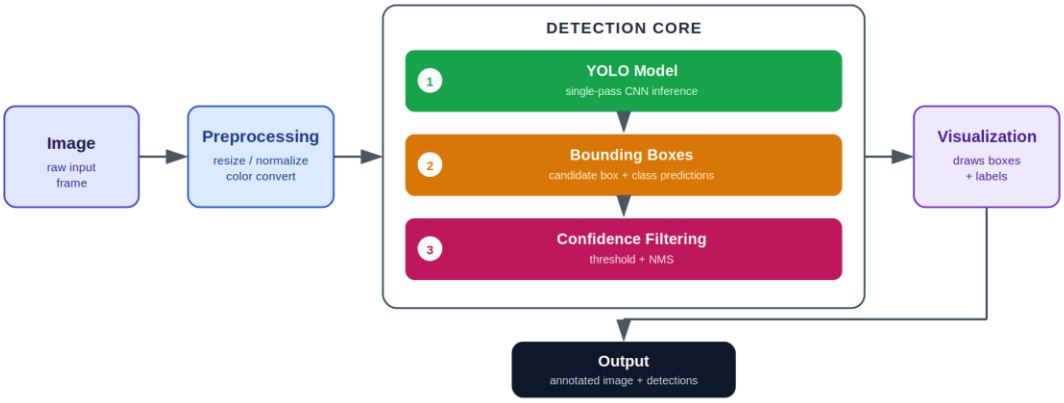


Figure 1. End-to-end architecture of the YOLO detection pipeline. The three numbered sub-stages inside the Detection Core execute strictly in sequence for every frame.

2. Image

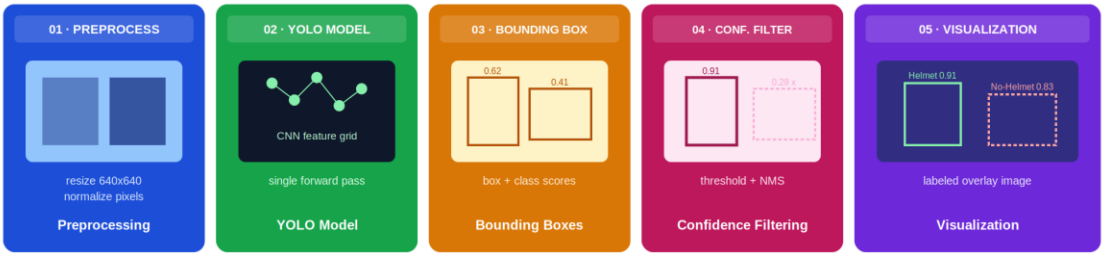
The pipeline begins with a single video frame or static image, captured from a webcam, an IP camera stream, or read from disk. This is the only external input; every later stage runs automatically without further intervention.

3. Preprocessing

The raw frame is resized to the model's input resolution, its pixel values are normalized, and its color channels are reordered so the tensor matches what the network was trained on.

4. Detection Core

This is the heart of the pipeline. A single frame tensor is transformed into a set of validated, labeled detections through three sequential operations, illustrated in Figure 2.



4.1 YOLO Model

The preprocessed tensor is passed through the trained YOLO network in one forward pass, producing a dense grid of candidate detections directly, without a separate region-proposal step.

```
results = model.predict(frame, imgsz=640)
```

4.2 Bounding Boxes

Each candidate detection is decoded into a box's coordinates, a predicted class (helmet or no-helmet), and a raw confidence score.

```
boxes, classes, scores = results.boxes.xyxy, results.boxes.cls, results.boxes.conf
```

4.3 Confidence Filtering

Detections below a confidence threshold are discarded, and Non-Maximum Suppression (NMS) removes overlapping duplicate boxes for the same object.

```
keep = scores > 0.5
```

5. Visualization

Surviving detections are drawn back onto the original frame as colored rectangles with class labels and confidence scores, giving a human-readable view of what the model found.

6. Output

The final annotated frame, along with optional metadata such as detection counts and per-class statistics, is displayed, saved, or streamed onward. This completes the pipeline and closes the loop from raw frame to interpreted result.

7. Pipeline Summary

Table 1 summarizes every stage alongside its representative operation and purpose.

Stage	Purpose
Image	Supplies the raw input frame
Preprocessing	Standardizes input for the network
YOLO Model	Single-pass CNN inference over the frame
Bounding Boxes	Raw box coordinates, classes, scores
Confidence Filtering	Removes weak / duplicate detections
Visualization	Draws labeled boxes on the frame
Output	Returns the final annotated result

8. Conclusion

This pipeline shows that real-time PPE compliance checking reduces to a disciplined sequence: prepare the frame, run one forward pass through YOLO, decode and filter the resulting boxes, and render the result. Overall accuracy depends on the weakest stage in the chain, poor preprocessing degrades detections, and a loose confidence threshold lets false positives through. Understanding each stage's individual role, as documented above, is essential to tuning and debugging the system as a whole.